Extropy Codex, Version 2.0

A Non-Extractive Contribution Ledger for Verified Entropy Reduction

Randall Gossett, with Perplexity Computer (Claude Fable 5), AI-assisted drafting disclosed

2026-07

Codex v2.0, formula canonical-v3.1.3, protocol v0.1

Abstract

The Extropy Engine is a protocol for measuring and rewarding verified reductions in disorder across eight domains of human and civilizational activity, using a single scalar unit called bits-equivalent (b_e) and a single canonical minting formula. Codex v2.0 supersedes v1.0 in full. It ships four architectural invariants that were absent, informal, or contradicted in prior drafts: non-extraction of value to external markets, cross-domain measurement normalization with a stated falsifier contract, a bounded settlement-time factor with a governance-set floor, and an implementation-agnostic protocol contract that separates the specification from the reference codebase. It also formally retires framings that v1.0 published, most notably the pseudo-physics form $XP = \Delta S / c\_L^2$. The result is a smaller, more disciplined document with sharper boundaries and honest falsification criteria. This Codex is the internal specification of the project. Its external contract is `PROTOCOL.md v0.1` in the same repository.

Reading Guide

This Codex has three registers.

Each section opens with a short plain-language paragraph that names what the section is trying to do and why it matters. Anyone comfortable reading a well-written policy memo can follow those paragraphs.

The body of each section is the technical register: formal statements, formulas, invariants, and references to specific files in the reference repository. That register targets protocol implementers, validators, governance participants, and reviewers.

The third register is the falsification register: what would have to be true, empirically, for the section's claims to be wrong. Falsification statements are called out inline and are the fastest way to attack the Codex on its own terms.

Throughout, references in the form `docs/FILENAME.md` refer to companion documents in the extropy-engine repository at `github.com/00ranman/extropy-engine`. Where this Codex and a companion document disagree, treat the Codex as the higher-order statement and the companion document as the current implementation-level detail. If the disagreement is substantive rather than stylistic, that itself is a bug and should be filed.

Methods Note: AI-Assisted Drafting

This Codex was drafted by Randall Gossett working with Perplexity Computer, using Claude Fable 5 as the underlying model, against the live repository at `github.com/00ranman/extropy-engine`. The assistant read the repository's source documents (`PROTOCOL.md`, `NORMALIZATION.md`, `NON_EXTRACTION.md`, `REJECTED_FRAMINGS.md`, `SPEC_v3.1.md`, `GAPS.md`, `cartel-threshold-analysis.md`, `VALIDATION_IS_EMERGENT.md`, `THREE_LAYER_SEPARATION.md`, `PSLL.md`, `IDENTITY.md`, `QUEST_MARKET.md`, `GOVERNANCE_DEFAULTS.md`, and `AUTARKY.md`), produced structured drafts, and iterated with the author on framing, technical accuracy, and register. All architectural decisions, the choice of what to retire from v1.0, and every substantive claim are the author's. The assistant's role was drafting, cross-checking claims against repository state, and structural editing.

This disclosure is included because Codex v2.0 will be published to Academia.edu, and scholarly reviewers deserve to know how the document was produced. It is not a disclaimer. Every claim in this document is the author's and is either supported by a specific file in the repository, by a cited reference, or by an explicit conjectural marker.

1. What the Extropy Engine Is

The Extropy Engine is a contribution ledger. It records that a specific reduction in disorder happened, in a specific domain, verified by a specific validator neighborhood, and it mints a non-transferable access threshold called XP against that record. It is not a currency, not a market, and not an economy in the extractive sense. It is a metering layer over verified work.

In one sentence: the Extropy Engine measures verified entropy reduction and translates that measurement into what a participant is currently admitted to do inside the protocol.

The protocol is defined by five commitments, restated from `docs/SPEC_v3.1.md` with the v3.1.3 corrections applied:

1. Value is measurable as entropy reduction, in bits-equivalent (b_e), under a domain-specific measurement operator M_d.
2. Entropy is measurable across eight domains: thermodynamic, informational, social, economic, ecological, governance, cognitive, and a reserved spiritual slot with no accepted M_d.
3. Intelligence stays at the edge. Each participant runs their own model or model consensus locally. The shared network is a coordination and accounting layer, never a supermind.
4. Verification is adversarially robust, privacy preserving, and incentive aligned, and it is performed by contributors doing contribution tasks, not by a separate validator class.

5. Governance is fractal, composable, and bounded against permanent concentration, with every operational parameter tunable through the same loop mechanism that mints XP.

The remaining sections of this Codex are the operational statement of those five commitments.

Falsifier for this section: if any of the five commitments is contradicted by shipped protocol behavior, the affected commitment must be either restated to match reality or the protocol must be corrected. A commitment that the protocol does not enforce is not a commitment. It is marketing.

2. What Changed From v1.0

Codex v1.0 was assembled in May 2026 as a synthesis of the companion book *Unfuck the World for a Dollar*, the v3.1 technical specification, an earlier peer-review response, and the Signal representational-fidelity paper. It was 113 pages. It served its purpose as a snapshot of the project's thinking at that moment. Four architectural facts have changed since then, and v1.0 does not reflect any of them cleanly.

2.1 Non-extraction is now an architectural invariant, not an open question.

v1.0 treated the question of whether XP or CT could bridge to fiat as an open regulatory concern to be resolved later. Codex v2.0 closes it: the protocol has no fiat bridge, no CT-to-fiat surface, no transferable wrapper, and no prediction market over loop outcomes, at any layer, ever. XP and CT are stateful access thresholds, not balances. This is not a policy decision, it is a design invariant of the same order as Digital Autarky. The full statement lives in `docs/NON_EXTRACTION.md` and is restated in §5 of this Codex.

2.2 Cross-domain ΔS is now a single common unit with a stated falsifier contract.

v1.0 spoke of ΔS in "bits" across eight domains as if that alone made the values comparable. Codex v2.0 introduces a single common unit, bits-equivalent (b_e), with a per-domain measurement operator M_d and five invariants that any M_d must satisfy. It also states four falsification conditions (F1 through F4) under which the whole project is falsified as a coordination protocol. If a principled M_d family cannot be constructed across adopted domains, or if validator disagreement dominates ΔS variance, or if cross-domain arbitrage cannot be closed by governance parameters, the affected domains are removed from the mint layer. The full statement lives in `docs/NORMALIZATION.md` and is restated in §7 and §8.

2.3 The settlement-time factor is now bounded.

v1.0 published the XP formula with a raw `log(1/Tₛ)` term. In the reference implementation through v3.1.2, that expression was applied to raw wall-clock seconds, which produced two simultaneous bugs: sub-second closures diverged the log term toward infinity, and any realistic closure over one second produced a negative log that was silently clamped to zero, then papered over by a rejected fallback branch. Codex v2.0 publishes the corrected formula: T_s is normalized as `exp(-λ · Δt)` and clamped into `(T_floor, 1]`, and the log-decay term is capped at `log(1/T_floor)`. Default `T_floor = 0.01` yields a hard cap near 4.605. This is the actual anti-speed-farming invariant. The bug and the fix are documented in `docs/CHANGELOG.md` under v3.1.3, and the formula is implemented in `@extropy/xp-formula` (`packages/xp-formula/src/index.ts`), stamped `canonical-v3.1.3`.

2.4 Specification and reference implementation are now separated.

v1.0 was ambiguous about whether the TypeScript reference implementation was the specification or a specific realization of it. That ambiguity meant any bug in the reference impl looked like a bug in the theory. Codex v2.0 draws the line explicitly: the specification is `docs/PROTOCOL.md v0.1` (the external contract), plus this Codex (the internal statement). Any implementation in any language that satisfies the protocol contract is Extropy-conformant. The TypeScript packages in this repository are the current reference implementation, not the specification. This distinction is elaborated in §19 and §20.

2.5 Retired framings are enumerated.

v1.0 taught XP = ΔS / c_L² as an "irreducible" physics floor. Codex v2.0 retires that framing. The formal retirement, with reasoning, lives in `docs/REJECTED_FRAMINGS.md` R1 and is summarized in §22. Two other framings are marked for retirement pending review: the phrasing of "cryptographic loop closure" as a security guarantee (R2) and the v4-6 trilemma phrasing as a proved theorem (R3). Both are currently open in `REJECTED_FRAMINGS.md`.

Codex v2.0 does not include a retired-framings preservation appendix. Retired framings live in `REJECTED_FRAMINGS.md` and are cited from this Codex, not embedded. A reader who wants to know what the project used to say and no longer says should read that document.

3. The Non-Extraction Invariant

Extropy tokens are non-transferable access thresholds. They are never balances to be extracted.

The formal statement, from `docs/NON_EXTRACTION.md`:

> *There is no path in the protocol, at any layer, by which an actor converts an XP or CT balance into a claim on fiat, another cryptocurrency, or any external transferable value, whether directly or through wrapped, synthetic, derivative, or off-protocol side-market representations.*

This is an architectural invariant of the same order as Digital Autarky. If a feature can be added without breaking Autarky, and can be added without breaking Non-Extraction, it is allowed. If it breaks either, it is not shipped.

3.1 The two failure modes ruled out simultaneously

The Howey trap. Any token that is purchasable, transferable, and held with the expectation of profit derived from the work of others becomes a security under United States case law. Every prior iteration of a protocol like this that added a "small liquid market" has ended in one of two places: it registered as a security and was captured by the same intermediaries it was designed to route around, or it operated in the grey and eventually stopped operating. The way out is not a better lawyer. The way out is a token with no cash-out surface.

The Nash flip. The feedback loops in this Codex rest on a Signal equation whose equilibrium is honest signalling when the payoff for gaming is bounded by what the loop itself produces. The moment loop output can be sold outside the protocol, the payoff for gaming becomes the external market price, which is unbounded from the loop's perspective. That flips the equilibrium. Cartels form because the return on colluding to inflate ΔS scales with an out-of-loop price rather than with in-loop access.

Both failure modes disappear if the token has no external price.

3.2 Access-economy semantics

Under Non-Extraction, XP and CT are not “how much I have” but “what I can currently do”.

- **XP** is a stateful threshold. Above a threshold τ_action, the actor is admitted to a class of actions: validating in a domain, opening a loop of a given rarity, joining a validator neighborhood. Below τ_action, the action is not admitted. XP is not spent by taking the action.
- **CT** is a per-actor structural coefficient in the Signal equation. It shapes access weight in ties and vote weight in validation. CT is not spent by voting.
- **Consumption** is metered against a contribution/draw ratio ρ = ΔS_produced / ΔS_consumed, computed on the ledger itself. If ρ drifts below the domain-set floor ρ_min_d, access degrades automatically. Restoration of ρ is itself a mintable loop.

The ratio ρ is what a fiat balance would have measured if we had one. The point of the architecture is that we do not need one to enforce reciprocity, because reciprocity is a property of the actor’s own ledger history.

3.3 The three tests every feature must pass

Before any feature merges into the protocol layer, it MUST pass all three of the tests below. The full text lives in `docs/NON_EXTRACTION.md` §4. The condensed statement:

T1. Extraction test. Can any actor convert an on-protocol balance into an off-protocol transferable value, directly or through a wrap? If yes, T1 fails.

T2. Counterparty test. Does the feature introduce a stable second party whose sole role is to pay for XP or CT? If yes, T2 fails.

T3. Ratio test. Does the feature preserve the contribution/draw ratio ρ as the primary throttle, or does it introduce a shortcut that lets an actor draw without proportionately producing? If yes, T3 fails.

Every module in `packages/xp-mint`, `packages/reputation`, `packages/loop-ledger`, and any future redemption package MUST include tests that exercise these three tests against its public surface.

3.4 Prediction markets over loop outcomes

Prediction markets are the canonical “just add a small liquid market” idea. Under Non-Extraction they are explicitly out of scope at the protocol layer. Domain-internal forecasting to help validators calibrate is not a prediction market for the purposes of this document. The distinguishing feature is whether the payoff is a transferable, tradable claim. Validator calibration payoffs that are non-transferable access thresholds are compatible with Non-Extraction.

3.5 What Non-Extraction is not

It is not a claim that no one will ever try to build an off-protocol market in XP receipts. People will. The design goal is that such a market has no counterparty support inside the protocol, so it stays a fringe activity and never captures the equilibrium.

It is not asceticism. Actors convert access thresholds into real-world value all the time by using them. What they cannot do is package unused access as a transferable claim.

It is not a substitute for legal review. It removes the strongest known legal attack surface (Howey) but does not preempt all others.

Falsifier for this section: if any shipped protocol feature passes review while failing any of T1, T2, or T3, either the test framework is broken or the feature is a violation. Either resolution requires a public correction. Non-Extraction is falsified as an invariant the moment it is treated as guidance.

4. Digital Autarky

Digital Autarky means every participant remains sovereign over their own intelligence stack, identity material, decision context, and local event history. The network is a coordination and accounting layer, not a supermind. The full statement lives in `architecture/AUTARKY.md`. This section restates the operational content.

The concern this principle addresses is a specific failure mode: centralized intelligence becomes a hidden control point. A network that decomposes reality on behalf of its users, even with the best intentions, eventually shapes what users perceive, what they can claim, and what counts as valid. That control surface is what every prior coordination platform has converged toward regardless of starting ideology. The Extropy Engine refuses that surface not as a stylistic choice but as a structural commitment.

4.1 Who controls what

Layer	Sovereign holder
Personal AI / model selection	The participant
Local raw context (private logs, sensors, observations)	The participant
Identity material (KYC artifacts, biometric bindings)	The participant’s device
Decision history (PSLL)	The participant
Claim formulation (decomposition into actionable units)	The participant’s personal AI
Submitted claim payload	The minimum interoperable surface the network needs
Validation outcomes	The mesh, via incentive-aligned peer review
Receipts and DAG entries	The shared network (immutable, public)

4.2 What the network is allowed to do

Standardize claim and quest schema. Route claims to validator neighborhoods via SignalFlow. Operate the validation lifecycle. Mint, burn, and settle XP via the canonical formula. Record receipts to the DAG. Aggregate and surface emergent epistemology (see §11). Enforce shared protocol rules (anti-Sybil, governance thresholds, decay).

4.3 What the network is not allowed to do

Decide what a participant meant by their input. Decompose claims for users. Hold a private world model over participants. Require raw personally identifiable information or full local logs. Operate a central brain that the rest of the system depends on. Establish a single source of epistemic authority.

4.4 The claim package

The handshake between participant and network has a fixed shape. Personal AI sends a Claim Package; network returns routing and validation receipts. The Claim Package contains a schema-conformant claim (domain vector E, falsifiability score F, ΔS estimate in b_e, evidence pointers), an identity proof (ZKP from the Identity layer), a PSLL anchor reference (a Merkle commitment, never the raw log), and optional quest-market metadata if the claim originates from an accepted quest.

The network does not receive raw conversational context with the personal AI, full PSLL contents, identity material beyond what the ZKP proves, or the decomposition reasoning behind the claim. It receives only the resulting claim.

Falsifier for this section: if any shipped protocol service accepts raw personal context, raw PSLL contents, or the participant's private reasoning as input, Autarky is falsified at that service. The service must be reduced to the Claim Package surface or removed from the protocol layer.

5. The Eight Domains and the B_e Common Unit

The Extropy Engine recognizes eight domains in which entropy appears in human systems. Every valid contribution claim must reduce entropy in one or more of these domains, and the reduction must be expressed in a single common unit before it enters the XP formula.

That single common unit is bits-equivalent (b_e). All domain-native ΔS measurements MUST be converted to b_e before being fed to the XP formula. The conversion is a domain-specific measurement operator M_d applied to the raw evidence and the domain state:

```
ΔS_bₑ = M_d(raw evidence, domain state)
```

Each domain owns its own M_d. The invariants any M_d must satisfy are listed in §7.

5.1 The eight domains

Note on domain nomenclature. Codex v1.0 used a slightly different set of domain labels in `docs/SPEC_v3.1.md` §5 (cognitive, code, social, economic, thermodynamic, informational, governance, temporal). The normalization document `docs/NORMALIZATION.md` uses the set below (thermodynamic, informational, social, economic, ecological, governance, cognitive, spiritual). Codex v2.0 adopts the `NORMALIZATION.md` set because it is the set the mint layer normalizes over. The `SPEC_v3.1.md` labels remain useful as operational categorizations of contribution style, but the canonical mint-side domains are the eight below.

Thermodynamic. Physical disorder. 1 b_e_thermo ≡ one bit of thermodynamic entropy erased, which by Landauer's principle corresponds to `k_B T ln 2` joules of minimum dissipation avoided.

Informational. Disorder in records, data, channels, and archival coherence. Anchor is raw Shannon bits with $H_0 = 1$. Prediction-loop instances (proofs, patches merged) use log-likelihood-ratio of the claim under the pre-verification distribution.

Social. Disorder in coordination. 1 b_e_social ≡ removing one round of "who does what" ambiguity in a coordination game.

Economic. Disorder in allocation. 1 b_e_econ ≡ resolving one bit of allocation-outcome uncertainty in a market or matching. Never denominated in fiat.

Ecological. Disorder in ecosystem state. 1 b_e_eco ≡ resolving one bit of ecosystem-state uncertainty over species, stocks, or flows in a bounded region.

Governance. Disorder in decision systems. 1 b_e_gov ≡ removing one bit of decision uncertainty under a codified rule set.

Cognitive. Disorder in belief states. 1 b_e_cog ≡ removing one bit of belief-state uncertainty in a verifiable epistemic agent (test scored, model calibration improved, misconception corrected in a way validators can inspect).

Spiritual. Not adopted. Retained as a domain label in the `EntropyDomain` enum for schema stability. There is no accepted M_spiritual, so this domain MUST NOT mint XP under the canonical formula until such an operator is defined, tested, and adopted through governance. Treat as a reserved slot.

5.2 Why a common unit at all

The mint layer needs a scalar. XP is a single number. If domains contributed incommensurable units, the ledger would be an arithmetic on category errors dressed as physics. The b_e commitment is precisely the commitment to make the arithmetic honest, or to face the fact that it cannot be made honest and remove the affected domains. §8 states the conditions under which the removal is forced.

5.3 Intersectionality

Real contributions often land across multiple domains at once. A teacher may reduce cognitive, social, and possibly temporal disorder in one act. A clean software deployment may reduce informational, economic, and possibly ecological disorder. The engine does not force a claim into one exclusive box. It measures a domain vector E and weights it contextually through the vector w (see §9).

6. Cross-Domain Normalization and the Falsifier Contract

The commitment to a b_e common unit is not free. It requires that eight measurement operators M_d exist, that they produce values inside a shared numeric envelope, and that those values track a real, verifiable, reproducible quantity in each domain. This section states the invariants those operators must satisfy and the conditions under which the whole architecture is falsified.

6.1 Invariants any M_d must satisfy

1. **Non-negativity.** $M_d(x, y) \geq 0$ for all (x, y). $\Delta S \leq 0$ does not mint.

2. **Boundedness per event.** M_d has a documented per-event cap that scales with the size of the underlying state space, not with wall-clock time. Time-based scaling belongs in the settlement-time factor, not in ΔS.
3. **Verifiability.** M_d is computable from evidence a validator neighborhood can independently reproduce. If the evidence cannot be independently reproduced, the mint MUST fail preconditions.
4. **Deterministic given evidence.** Two validators applying M_d to the same evidence produce values within a documented tolerance ε_d.
5. **Composability.** M_d is additive over disjoint sub-events. If a single logical loop is split into k sub-events, sum of ΔS_b_e over the k sub-events equals ΔS_b_e of the whole, up to ε_d.

6.2 The four falsification conditions

The Extropy Engine is falsified as a coordination protocol if any of the following holds, project-wide.

F1. Non-comparability. No principled M_d family can be constructed such that the invariants in §6.1 hold across every adopted domain simultaneously.

F2. Runaway cross-domain arbitrage. Given constructed M_d, there exists a strategy that repeatedly converts low-cost b_e in domain A into high-value XP redeemable against domain B's access thresholds, at a rate that cannot be closed by governance parameters (rarity R, frequency-of-decay F, settlement-time floor T_floor).

F3. Validator disagreement dominates signal. For at least one adopted domain, validator-to-validator variance on M_d applied to the same evidence exceeds the mean ΔS_b_e per loop. In that case, the mint is dominated by validator noise, and the ledger is measuring reviewer opinion, not entropy reduction.

F4. T_floor arbitrage after clamp. Even with the v3.1.3 T_s floor in place, there exists a domain in which the achievable log-decay term at settlement-floor speed is a k× multiple of the domain's honest median log-decay, where k is large enough that the resulting mint dominates the (R × F × ΔS × wE) factor for legitimate loops in the same domain.

If any of F1 through F4 is empirically confirmed and remains open for two consecutive governance cycles, the project MUST publish that fact, mark the affected domain(s) inactive at the mint layer, and stop claiming a physics-grounded XP invariant across those domains.

This is intentional. A framework that cannot be falsified is not a framework, it is a brand.

6.3 What v3.1.3 actually delivers

The v3.1.3 release does not solve the normalization problem. What it does deliver:

- A single common unit (b_e) with a canonical name and precise per-domain semantics, so future violations of §6.1 can be pointed at unambiguously.
- A settlement-time floor T_floor and log-decay cap in `@extropy/xp-formula`, closing the raw-seconds pathology that had made F4 trivially satisfiable in the reference implementation.
- Removal of the $XP = \Delta S / c\_L^2$ framing, which pretended to define an "irreducible XP" using a domain constant c_L but did not respect §6.1 (it collapsed all domains to their causal-closure speeds, which is a property of the substrate, not of the entropy delta).
- An explicit falsification contract in §6.2.

Every future domain adoption MUST include a proposed M_d and a validator-neighborhood recipe. No M_d, no mint.

7. The Canonical XP Formula

The canonical XP formula, at version `canonical-v3.1.3`:

```
XP = R × F × ΔS_bₑ × (w · E) × min(log(1/Tₛ), log(1/T_floor))
Tₛ = exp(-λ_d · Δt)                  Δt in seconds, λ_d per-domain
Tₛ ∈ (T_floor, 1]                    by construction
```

The formula is implemented in `packages/xp-formula/src/index.ts`. Every service that mints XP imports from there. No reimplementations. The formula-version string is stamped onto every mint event, and a mismatch between the stamped version and the executed math is a critical bug that MUST fail closed.

7.1 Variable meanings and ranges

Variable	Range	Meaning
R	[0.1, 10.0]	Rarity coefficient. Domain-specific. Governance-tunable.
F	[0, 1]	Falsifiability score. How testable the claim is.
ΔS_b_e	[0, cap_d]	Entropy reduction in bits-equivalent under M_d. Bounded by domain per-event cap.
w	[0, 5]^8	Domain-weight vector. Governance-adjustable per DFAO.
E	[0, 1]^8	Eight-domain entropy vector. Measured per claim.
T_s	(T_floor, 1]	Normalized settlement-time factor. Never raw seconds.
T_floor	(0, 1)	Governance-set settlement-time floor. Default 0.01.
λ_d	> 0	Per-domain settlement-decay constant.

7.2 What each factor does

R (Rarity). R weights claims by how rare the closing move is in the domain. It is not reputation. It is not history. Rarity is a property of the move, not of the actor. Past actions never inflate R for new claims. The R = reputation confusion was retired in v3.1.2; see the R rules restatement in `docs/THREE_LAYER_SEPARATION.md`.

F (Falsifiability). F is the falsifiability score of the claim. High-F claims have explicit disproof conditions, measurable evidence paths, and clear settlement criteria. Low-F claims are vague, subjective, or weakly instrumented. The engine does not forbid low-F claims. It refuses to pretend they are equal to claims that can actually be tested.

ΔS_bₑ. The measured entropy reduction, in bits-equivalent, produced by applying M_d to the evidence. This is the term §5 and §6 exist to make honest. If no meaningful ΔS can be measured, the mint fails pre-conditions.

(w · E). The dot product of the governance-set domain weight vector w and the claim's measured domain vector E. This is where contextual nuance enters without abandoning a universal metric. A software-oriented DFAO may weight code (informational) and governance heavily. A neighborhood repair DFAO may weight social and ecological differently. The domain-weight vector w is a per-DFAO override on top of ecosystem-level defaults.

min(log(1/Tₛ), log(1/T_floor)). The bounded settlement-time factor. This is where the v3.1.3 correction lives. T_s is `exp(-λ_d · Δt)`, which is monotonically decreasing in elapsed time, has value 1 at Δt = 0, and asymptotes toward 0 as Δt grows. The clamp `Tₛ ∈ (T_floor, 1]` guarantees the log is bounded on both sides: it cannot diverge to +∞ at sub-second closures (which would allow speed-farming), and it cannot go negative for slow closures (which would silently zero legitimate mints, which was the actual bug in v3.1.2). With `T_floor = 0.01`, the log-decay cap is `log(100) ≈ 4.605`.

7.3 What the formula does not contain

Reputation is not in the mint formula. Reputation is a property of the actor over time, and it lives in CT and in the actor's reputation vector; it is not a multiplier on new XP mints. This is a load-bearing separation. If R could be inflated by reputation, then reputation itself would compound and produce a run-away.

The XP = ΔS / c_L² branch is not in the formula. It was retired in v3.1.3 and is documented in `docs/REJECTED_FRAMINGS.md` R1. The `IrreducibleXPInputs` type is kept in `packages/contracts` with a `@deprecated` marker for one release, then removed.

Fiat is not in the formula. It cannot appear. See §3.

7.4 The distribution rule

XP is distributed on closure. The exact split among validators (as loop closers) and the loop opener (as actor) is a governance parameter. The distribution rule MUST NOT set validator share to zero (that would kill validator incentive) and MUST NOT set actor share to zero (that would kill loop-opener incentive). See `docs/PROTOCOL.md` §7.

Falsifier for this section: if any shipped mint path executes math that differs from the version stamped on the mint event, formula-version drift has occurred and the deploy is invalid. The mint pipeline MUST fail closed on stamp mismatch. This is tested in `packages/xp-formula/src/index.test.ts`.

8. Loops: Lifecycle and Evidence

Every unit of value in the Extropy Engine is minted from a loop. A loop is a bounded piece of work that opens, gathers evidence, is scored by a validator neighborhood, and closes with a verdict. If the verdict is favorable, XP is minted against the loop record. If not, the loop is rejected and nothing is minted. There is no partial credit for unclosed loops.

8.1 The lifecycle

A loop passes through the following states, in order, with no retro-mutation:

```
proposed → open → evidence-submitted → validator-verdicts-collected → closed | rejected
                                                                          ↓
                                                                       minted → confirmed |
burned
```

Each state transition is a substrate-recorded event. Timestamps are wall-clock but not authoritative for XP; the authoritative time input is `elapsed_seconds = t_close − t_open` measured at substrate resolution. See `docs/PROTOCOL.md` §3.

8.2 The evidence requirements

Every loop closure MUST carry evidence that satisfies three conditions:

E1. Independently reproducible by any validator in the neighborhood, without contacting the actor.

E2. Domain-native. The domain's measurement operator M_d applied to this evidence yields ΔS_b_e within tolerance ε_d.

E3. Tamper-evident. A validator can detect if the evidence has been modified between submission and verdict.

Implementations may use content-addressed storage, signed attestations, or on-chain hashes to satisfy E3. The protocol requires only that E3 holds.

8.3 Validator verdicts

Each validator in the neighborhood emits a verdict:

```
verdict = { validator_id, deltaS_measured, decision ∈ {accept, reject, abstain}, evid-
ence_hash }
```

- **V1. Accept** iff `|deltaS_measured − deltaS_claimed| ≤ ε_d`.
- **V2. Reject** iff the difference exceeds ε_d, or any of E1 through E3 fails.
- **V3. Abstain** iff the validator lacks the sub-domain competency to score this loop.

Abstentions do not count in the closure quorum but MUST be recorded so that validator drift and coverage gaps are observable.

8.4 Closure

A loop closes when three conditions are met simultaneously. The number of `accept` verdicts satisfies the domain's quorum function `Q_d(N_participating)`. No `reject` verdict from a validator in the top decile of the neighborhood by CT stands unrebutted after the domain's rebuttal window. All accepts agree on ΔS within ε_d.

The mint uses `ΔS_final = median(deltaS_measured over accepts)`. Median rather than mean, so that a single outlier accept cannot inflate the mint.

8.5 Provisional mint and retroactive settlement

Mints are provisional until a retroactive-validation window closes. During that window, any validator (not just the original neighborhood) can submit a challenge. A challenge that produces a rebuttal accepted by ≥ Q_d validators burns the mint. The distribution is reversed. The formula-version stamp is retained so an audit can distinguish burns of legitimate mints (from ambient noise) from burns of gamed mints.

Retroactive confirmation follows the same mechanism in reverse: a provisional mint that survives its window with no accepted challenge is confirmed, and its XP is committed to the actor's thresholds.

8.6 Rejection

A loop is rejected outright, before mint, if any of the following holds: quorum is not reached, evidence fails E1–E3, all accepts do not agree on ΔS within ε_d, or ΔS_final ≤ 0 (see §6.1 invariant 1). Rejection is recorded on the substrate but produces no mint event.

Falsifier for this section: if any deployed loop pipeline permits state transitions other than those listed in §8.1, or mints XP without passing the closure conditions in §8.4, the pipeline is non-conformant and must be corrected.

9. Validation as Emergent Property

There is no validator class. There are only contributors performing entropy-reducing tasks. Some of those tasks happen to validate other tasks, and the person performing them often does not know it.

This is a load-bearing clarification. The words "validator" and "validation pipeline" appear throughout this Codex and the reference implementation, and it is easy to picture a separate class of people whose job is to sit in judgment of contributions. That picture is wrong, and it imports exactly the failure mode the protocol exists to remove. The full statement lives in `docs/VALIDATION_IS_EMERGENT.md`.

9.1 The misread

The intuitive model of any review system is two tiers: contributors do the work, validators check the work. That split creates a privileged class. Validators become a chokepoint, a target for capture, and a source of their own information entropy, because now you have to validate the validators. Every system that builds a dedicated review tier eventually has to answer "who watches the watchers", and the answer is always another tier, which generates the same problem one level up.

9.2 What actually happens

Validation is just another contribution task. It reduces disorder about the state of a claim: before the task, the claim's correctness is uncertain (high entropy); after it, the claim is confirmed or contradicted (lower entropy). That is entropy reduction by the same definition the whole protocol runs on. It mints XP the same way, decays the same way, and settles retroactively the same way as any other contribution.

Because validation is a task, it is performed by contributors. There is no separate population. The same person who writes code on Monday scores a blind slice on Tuesday and, on Wednesday, completes a quest whose output silently confirms or contradicts a third party's earlier claim. All three are entropy-reducing tasks. None of them carry a special validator badge.

9.3 Blind and implicit validation

Most validation in the mesh is not someone explicitly clicking approve. It is implicit and frequently invisible to the performer. Two mechanisms produce this.

Blind slicing. A claim is split into 1/10th slices and routed to contributors who see only their slice, not the parent claim or who made it. A contributor scoring a slice does not know whose work they are checking, or sometimes even that the slice belongs to a larger validation at all. They are performing a small entropy-reducing task. The aggregation layer turns their independent slice scores into the falsifiability signal for the parent. The validation is real; the performer's awareness of it is not required.

Downstream task overlap. Many tasks validate earlier tasks as a side effect of doing their own job. If task B builds on the output of task A, then B succeeding is partial confirmation of A, and B failing in a way traceable to A's output is partial contradiction of A. The person performing B is not "validating A". They are performing B. The contribution graph extracts the validation relationship after the fact, from the dependency structure.

9.4 Why this matters

No chokepoint to capture. Corporate capture in the threat model means a well-funded adversary employing real validators whose votes are externally directed. That risk shrinks when there is no validator class to employ. You cannot buy the review tier when the review tier is the entire contributor population doing ordinary tasks, most of them blind to what they are confirming.

No watcher regress. Because validation is a task that itself reduces entropy, it is subject to the same retroactive settlement as everything else. A validation that later proves wrong burns its XP and penalizes the reputation behind it, exactly like any other contribution that decayed. The watchers are watched by the same mechanism that watches everyone, so the regress terminates. There is no separate trust tier that has to be trusted axiomatically.

Goodhart resistance. When contributors do not know which of their tasks are validations, they cannot selectively perform for the validation. The admissibility condition for value is "did this genuinely reduce disorder", and a metric whose application point is hidden from you cannot be selectively optimized against. Blind and implicit validation is a structural anti-Goodhart property, not a policy.

9.5 The epistemology engine, correctly framed

The `epistemology-engine` package is the mesh's emergent peer-review witness layer. It is not a review service. It does not assign validators. It observes the task graph and reads validation out of it as an emergent property.

What it does: aggregates validation outcomes across the mesh and surfaces consensus drift, dissent clusters, and contested-claim patterns; computes mesh-wide falsifiability statistics (F-distributions per domain, per DFAO); tracks reputation graph evolution and exposes Sybil-suspicious clusters; surfaces emergent ontologies (recurring claim patterns, naming convergence, instrument standardization across DFAOs); provides queryable hooks for governance proposals.

What it does not do: decide what is true, perform claim decomposition, arbitrate disputes, or own a private world model.

Read the engine's role as: peer review is what the graph already is, not a step bolted onto it. Multiple instances of the engine can run independently. There is no canonical engine instance, by design.

Falsifier for this section: if any shipped protocol behavior admits a persistent, self-identified validator class that cannot itself be validated by contributors doing ordinary tasks, the emergence property is broken. Either the class is dissolved into ordinary contribution, or the emergence claim is dropped.

10. Reliability, Falsifiability, and Reputation

Three quantities in the protocol are sometimes conflated in casual discussion and MUST be kept separate in implementation. They enter different formulas, at different times, and answer different questions.

Reliability, expressed through the coefficient R in the XP formula. Rarity of the closing move in the domain. Governance-tunable. See §7. R is a property of the move, not of the actor.

Falsifiability, expressed through the score F in the XP formula. How testable the claim is. High F means explicit disproof conditions, measurable evidence paths, clear settlement criteria. Low F means vague, subjective, weakly instrumented. F is a property of the claim, produced during evidence submission and confirmed by the validator neighborhood.

Reputation, expressed through CT and through per-domain reputation vectors, never through R. Reputation is a property of the actor over time. It is computed from the ledger's own history and it decays. It shapes access weight in ties (through CT) and it gates validator neighborhood participation (through per-domain thresholds), but it never enters the mint formula. Past actions do not inflate new XP mints. If they did, reputation would compound, and the protocol would concentrate.

The three-way separation is stated as a hard rule in `docs/THREE_LAYER_SEPARATION.md`:

> *R is Rarity, not reputation. Past actions never inflate new XP. F is Frequency-of-decay. Not falsifiability. Not feedback closure strength. Not vote share. ρ (reputation density) lives only in CT. Reputation enters one place, not everywhere.*

Note the naming collision. `docs/THREE_LAYER_SEPARATION.md` uses F to mean Frequency-of-decay, while `docs/NORMALIZATION.md` and `docs/PROTOCOL.md` use F to mean Falsifiability. Codex v2.0 uses F to mean Falsifiability, consistent with the normalization document and the current protocol contract. The

frequency-of-decay factor is retained in the formula (it penalizes bursts of same-class loops from the same actor), and this Codex names it F_freq to disambiguate. The reference implementation is being reconciled to the F = Falsifiability convention; see `docs/GAPS.md` #25 for the vocabulary-standardization work item.

Falsifier for this section: if any shipped mint path allows an actor's reputation to raise their new-mint multiplier, the separation is broken and the protocol must be corrected before further mints proceed.

11. Identity, PSLL, and Selective Accountability

The identity layer establishes strong Sybil resistance and selective accountability without exposing raw identity material to the network. The full statement lives in `docs/IDENTITY.md` and `docs/PSLL.md`.

11.1 Design constraints (non-negotiable)

Easy onboarding for normal humans. Strong resistance to one-person-many-identity abuse. No raw personally identifiable information exposure to the network DAG. Selective reveal under governance conditions. Compatibility with edge-native intelligence (personal AI handles identity locally).

11.2 The canonical flow

1. User signs in via OAuth or OpenID Connect using familiar credentials.
2. On-device KYC binding runs entirely on the user's device: identity document parse, liveness, biometric bind, or trusted issuer handoff. The network sees nothing at this step.
3. Personal AI generates a W3C DID and Verifiable Credential locally.
4. Credential is wrapped in ZKP material. Default scheme: BBS+ (selective disclosure friendly, smaller proofs). Alternate: zk-SNARK circuits for specific predicates (age, jurisdiction, etc.).
5. Network receives only: proof of uniqueness, proof of valid onboarding, per-context nullifier, public DID. Network does not receive: raw documents, full biometric material, real-world identity tied to DID.

11.3 Threshold reveal escrow

Under governance threshold, a DID can be linked back to enforceable real-world identity. The provisional default is 7-of-12 ecosystem validators holding a valid governance proposal with cause shown. Threshold-keyed escrow holds the reveal material (Shamir-style or threshold encryption). The threshold is tunable per ecosystem DFAO.

This is selective privacy under enforceable accountability. It is not anonymity, and it is not surveillance. The mesh cannot see who a DID is, and it cannot correlate DIDs across DFAOs without consent (nullifiers prevent that). Governance can pierce the veil with cause, through a public and auditable process.

11.4 The Personal Signed Local Log

Every participant maintains a Personal Signed Local Log, called PSLL, on their own device. The PSLL is append-only, hash-chained, cryptographically signed with the participant's DID key, locally controlled, and selectively disclosable. The pattern is borrowed from Holochain's source-chain concept and reimplemented natively.

The network does not ingest raw PSLL payloads. Instead, periodic Merkle-root commitment receipts are anchored into the DAG. Under dispute, subsets of the PSLL can be revealed with inclusion proofs or ZKP-based selective disclosure.

The minimum PSLL entry schema is defined in `docs/PSLL.md`. Every claim submitted, every validation performed, every quest accepted or completed, every decomposition step, every governance vote, every reputation update, and every reveal-consent record MUST be logged to the PSLL. Other participants' PSLL contents are never logged (each PSLL is single-author). Raw network-side state is never logged (the DAG is the canonical record for that).

11.5 What Sybil resistance requires

Identity in v0.1 requires:

I1. Actor identifiers are stable across sessions. **I2.** Two identifiers referring to the same natural or legal person are recognizably related by an on-substrate mechanism, so that Sybil load can be estimated. **I3.** The identifier space is compatible with W3C DID; a DID-shaped identifier MUST resolve without an out-of-band lookup.

I2 does not require a mapping to legal identity. It requires only that Sybil clusters are visible to governance. See `docs/PROTOCOL.md` §10.

Falsifier for this section: if the network's threat model requires legal identity mapping to close Sybil clusters, either the ZKP layer is inadequate or the threat model has been misspecified. Either resolution requires a public correction.

12. The Contribution Graph and the Quest Marketplace

The operational primitive of the Extropy Engine is the loop, and the operational surface where loops appear is the quest marketplace. The marketplace makes contribution legible: it turns real-world requests, complaints, opportunities, and system signals into small structured units of work that can be routed, performed, validated, and settled. The full statement lives in `docs/QUEST_MARKET.md` and `docs/CONTRIBUTION_GRAPH.md`.

12.1 The unit of work

The default task grain is 2 to 5 minutes. This choice is operational, not philosophical. Contribution economies fail when contribution units are too vague, too large, or too slow to verify. A 2 to 5 minute default makes onboarding low-friction (anyone can start with a 3-minute task), validation tractable (volunteer slices stay small), coordination meaningful (claims close fast enough to feel real), and gaming expensive (small per-unit rewards mean farming costs scale linearly with the number of loops).

Larger work composes from micro-quests. A complex task is a graph of small ones. There is no separate track for “big” contributions; big contributions are decomposed by the participant’s personal AI into runs of small ones, each of which closes as its own loop.

12.2 The lifecycle

```
Real-world input
    │
    ▼ Personal AI mediation
Structured micro-claim
    │
    ▼ Decomposition into 2–5 min tasks
Micro-quest published to marketplace
    │
    ▼ SignalFlow routing (see §12.4)
Quest accepted by participant
    │
    ▼ Work performed
Completion submitted with evidence
    │
    ▼ Volunteer micro-validation (1/10th blind slices)
Weighted consensus, then loop closure
    │
    ▼ XP minted (provisional)
Retroactive validation window
    │
    ▼ Settle or burn
Final XP credited or revoked
```

12.3 Dynamic reward escalation

Neglected work automatically gets higher reward weight until accepted. The provisional curve is linear from 1.0× to 3.0× base XP weight over 7 days, then logarithmic to a cap of 10.0×. On acceptance, the escalation resets to 1.0×. The curve is governance-tunable per DFAO.

12.4 SignalFlow routing

Each quest is routed based on a four-factor signal:

```
score = w_d × domain_match
      + w_r × reputation
      + w_l × current_load_inverse
      + w_a × historical_accuracy
```

Default weights `(w_d, w_r, w_l, w_a) = (0.35, 0.30, 0.15, 0.20)`. Per-DFAO override allowed. The routing function is a scoring rule, not an assignment. The participant retains refusal.

12.5 Validation by 1/10th blind slices

Default: validators see a 1/10th blind slice of a claim. Aggregation across slices produces F. Benefits: single-validator influence is diluted; privacy is preserved (no validator sees the full context); Goodhart-resistance improves (gaming requires coordinated capture across many validators); accessibility increases (1/10th of a small claim is a sub-minute task).

Full-context validation remains supported for high-stakes or low-decomposability claims. It is a fallback, not the default.

12.6 Anti-abuse

Decomposition checks: claims that cannot be decomposed below threshold task size are flagged.

Throughput rate-limiting: per-participant velocity caps prevent farming bursts. The specific caps are governance-tunable and are the main knob against F4 (see §6).

Reputation-gated escalation: acceptance of escalated-reward quests requires minimum domain reputation.

Cross-validation correlation: validators whose scoring correlates suspiciously closely are surfaced by the `epistemology-engine` as Sybil-suspicious clusters.

Falsifier for this section: if the marketplace's routing produces persistent geographic, demographic, or reputational underrepresentation that governance cannot close, the marketplace is not a coordination surface. It is a filter. Either the routing is corrected, or the coverage claim is dropped.

13. Substrate and the DAG

The Extropy Engine commits to a native substrate end-to-end. The engine is not deployed as an application on Holochain, Ethereum, Solana, or any other existing framework. Full Digital Autarky requires owning the lowest shared layer (handshake and DAG). Dependency on another project's plumbing would compromise the sovereignty commitment and create a supply-chain control point outside the network's own governance.

13.1 Borrowed patterns

Three architectural patterns are borrowed from Holochain and reimplemented natively, with credit given. The names are the Extropy names.

Holochain pattern	Extropy name	Purpose
Source chain	Personal Signed Local Log (PSLL)	Per-node append-only provenance
Neighborhood DHT	Validation Neighborhoods	Sharded validation load balancing and task discovery
Zomes / DNA modules	Rule Modules	Composable, fractal DFAO inheritance and evolution

Credit: the patterns are good. The implementations are the Extropy Engine's own.

13.2 The DAG

The shared substrate records loop state, validator verdicts, and formula-version-stamped mint events. It is append-only at the event layer, and it exposes the mint-event log to any actor. See `docs/PROTOCOL.md` §2.3 for the substrate role definition.

The DAG stores what everyone can see: standardized claim schema, routing envelopes, validation receipts, mint and burn events, PSLL Merkle-root anchors, reputation-graph updates, and governance proposals. The DAG never stores participant private context, raw PSLL entries, or identity material beyond the DID and its ZKP-wrapped credentials.

Causal edges in the DAG record dependencies between loops (loop B depends on the output of loop A). Those edges are what the `epistemology-engine` reads to surface downstream-task-overlap validation (see §9.3).

13.3 Substrate implementation

The reference substrate is the `packages/dag-substrate` module. Any alternative implementation is Extropy-conformant if it satisfies the substrate role definition in `docs/PROTOCOL.md` §2.3: append-only at the event layer, exposes the mint-event log, and correctly records the state transitions in `docs/PROTOCOL.md` §3.

Falsifier for this section: if the substrate exposes any mint event whose stamped formula version differs from the executed math, the substrate is non-conformant. Fail closed.

14. Three-Layer Separation

The Extropy Engine has three layers. They must stay separate. Confusing them is how every reputation system in history has consumed itself. The full statement lives in `docs/THREE_LAYER_SEPARATION.md`.

Layer	Visible to	Currency	Purpose
User-facing	Everyone	Discounts, savings, gamified feedback	Make participation feel good and pay off in real terms
Merchant-facing	Businesses	Better POS, customer pipeline, operational signal	Make merchants want in, without charging them SaaS
Engine	Validators, sensors, the math	XP, CT, and access thresholds	Actually measure entropy reduction. Never user-visible as a score.

Hard rule: Layer 1 never exposes raw XP as a number a user can target. Layer 3 never gets simplified into a public leaderboard. The gamification on Layer 1 is a deliberate decoy for user-facing psychology. The real scoring function lives where users cannot farm it.

14.1 Why this separation exists

Every gamified scoring system in history has been gamed. Credit scores become things people optimize for instead of actual creditworthiness. Klout became a parody of itself. FICO is a number people target, not a true measure. Steps, streaks, productivity dashboards: people fake the number, abandon the underlying behavior.

This is Goodhart’s Law: when a measure becomes a target, it stops being a good measure.

The standard response to Goodhart is to hide the metric. The Extropy Engine does that, with a twist: the metric is hidden not out of secrecy but because the user-visible gamification is deliberately a different metric from the system-level scoring function. The user can farm the visible one all day. It pays out in fun, dopamine, badges, streaks. It does not pay out in XP. XP comes from the underlying entropy-reduction signature computed on Layer 3 from validator-witnessed data. The user cannot push that lever directly because they do not know which lever it is, and even if they did, validators would catch the manipulation.

This is deliberate metric divergence between what the user sees and what the system rewards. The two are correlated by design. They are not the same.

14.2 What Layer 1 shows and hides

Shows: savings, gamified feedback, streaks, levels, badges, community achievements, character sheets curated by the participant. Hides: raw XP numbers, per-domain rarity multipliers, direct levers that map 1:1 to XP minting.

The character sheet metaphor: the participant holds a self-curated record of their habits and contributions. Validators and sensors prevent fabrication. The participant chooses what to display, but they cannot forge what is there. The participant holds the pen and the eraser. Reality holds the dice.

14.3 What Layer 2 offers merchants

A free or near-free point-of-sale that runs standard payment flows. A customer pipeline: participants preferentially shop at network merchants. Better operational signal than legacy key performance indicators (entropy-reduction patterns reveal what is actually working in the business). Standard merchant-services infrastructure at competitive rates. Optional DFAO node registration for deeper benefits.

Merchants do not see individual user XP balances, individual character sheets (unless the user shares), the full Layer 3 math, or any way to manipulate a customer’s score.

Merchants opt in because the POS is free or cheaper than what they were using, customers walk in asking which businesses are on the network, the operational data is better than legacy KPIs, and network-attracted customers have higher retention. The entropy-reduction layer is, from the merchant’s perspective, an invisible substrate that makes the business case work.

14.4 Monetization without extraction

Standard merchant-services fee capture (the fee the merchant was already paying someone else). DFAO node registration fees at scale. Treasury yield on protocol reserves. Premium analytics for businesses that want deeper signal. Multinational nodes paying for specialized integrations.

Not: SaaS subscriptions on participants. Not: paywalled POS. Not: user billing.

14.5 Hard rules at Layer 3

R is Rarity, not reputation. Past actions never inflate new XP.

F is Falsifiability (per this Codex and per `PROTOCOL.md` / `NORMALIZATION.md`). The frequency-of-decay factor is retained as F_freq; see §10.

ρ (reputation density) lives only in CT. Reputation enters one place, not everywhere.

Validators witness, they do not authorize. A participant cannot choose their validators. Validation is environmentally assigned by SignalFlow.

Falsifier for this section: if any deployed user surface publishes raw XP as a leaderboard, the Layer 1 / Layer 3 separation is broken. Either the surface is corrected, or the Codex's claim to Goodhart resistance is dropped.

15. Governance and the DFAO Model

Governance in the Extropy Engine is fractal, composable, and bounded against permanent concentration. The unit of governance is the DFAO (Digital Fractal Autonomous Organization), a rule-scoped organization that can nest inside other DFAOs, be nested by them, and evolve its own parameters through the same loop mechanism that mints XP.

15.1 The governance loop

Every parameter is a knob. Every knob has a default so the system runs. Every knob is votable. Nothing is locked. The provisional defaults, all governance-tunable, are enumerated in `docs/GOVERNANCE_DEFAULTS.md`:

Knob	Default	Vote tier
ZKP scheme	BBS+	Ecosystem
Identity reveal threshold	7-of-12 + cause-shown	Ecosystem
Reward escalation curve (early)	linear 1.0× to 3.0× over 7d	Domain DFAO
Reward escalation curve (late)	log to cap 10.0×	Domain DFAO
Retroactive validation window	30 days	Ecosystem
CT lockup	14 days	Ecosystem
Validator weight factors	4 (domain, rep, load, accuracy)	Ecosystem
PSLL anchor cadence	1 per loop close	Ecosystem
T_floor	0.01	Ecosystem
λ_d, R_d, ε_d, Q_d, ρ_min_d, τ_validator_d, τ_action_d	Per-domain	Domain DFAO
Quorum size formula	Open (see `GAPS.md` #1)	Domain DFAO

15.2 How a default changes

A personal AI drafts a proposal targeting the relevant tier. The proposal enters conviction voting in the appropriate DFAO. On passage, the new value is written to `governance/` and propagated. The PSLL records the proposal trail end to end. The mint pipeline stamps every mint event with the formula version and parameter set active at the time; the parameter change does not retro-apply to already-confirmed mints.

15.3 Fractal composition

DFAOs nest. A domain DFAO (e.g. informational) contains sub-DFAOs (e.g. a codebase-specific DFAO for one open-source project). Sub-DFAOs inherit parameters from their parent by default and may override them within governance-defined bands. Overrides that exceed the allowed band are rejected by the substrate.

Cross-DFAO governance conflicts (e.g. two sub-DFAOs disagreeing on a shared parameter) escalate to the nearest common ancestor DFAO. See `docs/GAPS.md` #40 for the escalation rules work item.

15.4 Anti-concentration

Reputation decays. CT decays. Domain weights are periodically re-normalized. Validator neighborhoods rotate. Retroactive validation windows let late correctors take back XP from confirmed-but-wrong closures. Every mechanism in this Codex that could produce concentration also produces an offsetting bleed. None of these bleeds is optional.

Falsifier for this section: if any DFAO can, through governance action alone, produce permanent concentration of validator power that cannot be undone by ordinary participant activity within a bounded number of cycles, the anti-concentration claim is falsified for that DFAO.

16. Cartel Analysis and Attack Surfaces

The Extropy Engine's threat model addresses adversarial validator behavior directly. The full analysis lives in `docs/cartel-threshold-analysis.md`. This section states the operational conclusions.

16.1 The cartel finding

Under the current XP, R, and validator parameters, collusive validator cartels are not a stable Nash equilibrium once cartel size $N \geq 10$. Defection (whistleblowing) dominates collusion because the retroactive-burn plus whistleblower-reward mechanism makes the one-time defection payoff strictly larger than the ongoing per-person collusion share.

Let X be the total XP produced by a collusive loop, N the cartel size, P_b the probability the loop is later burned (which grows with N), and R_defect the expected reputation gain from defecting. Per-person share if collude is `(X/N)(1 − P_b)`. Defection payoff is `0.50 X + R_defect`. Defection dominates when `0.50 X + R_defect > (X/N)(1 − P_b)`. With default parameters this holds for all $N \geq 10$, even without the explicit 50% whistleblower payout; the R_defect term alone is sufficient to flip the equilibrium.

16.2 Simulation results

Under an adversarial-loop simulation with an initial mix of 50% honest, 30% opportunistic, 20% cartel validators: the cartel win rate peaks near 61% around loop 2,500. Honest validators accumulate enough reputation to outvote the cartel after that. By loop 4,000, cartel membership drops from 20% to 3%. Sybil infiltration (a batch of 10 fake identities) triggers mass defection and collapses the cartel. About 78% of loops settle to honest validation. End-state reputation: honest average $R \approx 7.2$; cartel remnants $R \approx 3.1$.

16.3 Where the remaining attack surface concentrates

The main remaining surface is not collusion. It is the settlement-time factor T_s.

Before v3.1.3, T_s was speed-farmable: an attacker could shrink T_s toward zero to inflate `log(1/Tₛ)` and game XP issuance. The v3.1.3 T_floor and log-decay cap close the trivial version of this attack (see §7). What remains open is F4 (see §6): a domain in which the achievable log-decay term at T_floor is a large multiple of the honest median. The mitigation is governance-tunable per-domain λ_d and per-actor throughput rate limits. The residual risk is that any specific domain may need its λ_d and rate limits re-tuned in the presence of adversarial evidence.

16.4 Other surfaces enumerated in the reference implementation

Wash-loop detection across colluding identities. Bribery resistance under IT decay. Validator bid-rigging mitigation. Funded-validator (corporate-capture) defenses. CT lockup parameter optimization. Cold-start validator bootstrapping. Geographic and language balancing in SignalFlow. Adversarial-load shedding. Sybil-resistant load distribution under burst traffic. Cross-domain consensus weighting. Consensus failure recovery. All are in `docs/GAPS.md` P1 and P2 tiers.

Falsifier for this section: if adversarial simulation with the current parameters fails to converge to a majority-honest settlement over $N \geq 10$ cartel size within a bounded number of loops, the cartel analysis is falsified and either the parameters or the mechanism must be revisited.

17. Protocol v0.1 (External Contract)

The Extropy Engine specification and its reference implementation are separated by design. This Codex is the internal specification of the project. The external contract is `docs/PROTOCOL.md v0.1`. Any implementation in any language that satisfies the protocol contract is Extropy-conformant.

The protocol contract states, in short:

- **Roles.** Actor, Validator neighborhood, Substrate.
- **Loop lifecycle.** The state machine in §8.1 above, with substrate-recorded transitions and no retro-mutation.
- **Evidence.** E1 independently reproducible, E2 domain-native, E3 tamper-evident.
- **Verdicts.** V1 accept iff within ε_d, V2 reject iff outside ε_d or E1–E3 fails, V3 abstain iff sub-domain incompetency.
- **Closure.** Quorum function Q_d met, no top-decile reject stands unrebutted, all accepts agree on ΔS within ε_d. ΔS_final is median across accepts.
- **Mint.** The canonical formula, stamped with formula version, failing closed on version mismatch.
- **Non-transferability.** N1 no first-class XP transfer, N2 no XP-for-external-value exchange, N3 access thresholds are the only sanctioned mechanism for using accumulated XP.
- **Retroactive validation.** Provisional mints, challenge window, burn on accepted rebuttal.
- **Identity.** I1 stable identifiers, I2 Sybil clusters visible to governance, I3 W3C DID compatible.

- **Governance parameters.** R_d, λ_d, T_floor, Q_d, ε_d, τ_validator_d, τ_action_d, ρ_min_d, retroactive validation window length. Changes close as loops in the `governance` domain and do not retro-apply.
- **Conformance.** Implement §3 through §9 of the protocol; ship at least one domain M_d that satisfies `NORMALIZATION.md` §4 invariants; pass the three non-extraction tests; publish and enforce the formula version.
- **Non-goals.** Fiat bridges, cross-substrate migration, prediction markets over loop outcomes, tradable token wrappers.

The protocol contract is deliberately smaller than this Codex. It contains what every implementation MUST do. This Codex explains why, and it states additional invariants (Three-Layer Separation, emergent validation, the falsifier contract) that shape how a conformant implementation is designed.

18. Reference Implementation Boundary

The current reference implementation is the TypeScript codebase in this repository. It is not the specification. The following table maps the reference packages to the specification sections they realize.

Reference package	Realizes
`packages/xp-formula`	The canonical formula (§7). Stamped `canonical-v3.1.3`. Contains the property tests that bind the mint pipeline to §7's invariants.
`packages/xp-mint`	The mint pipeline (§7, §8). Routes every mint through `computeXPFromElapsedSeconds`. Stamps rows with the formula version.
`packages/loop-ledger`	Loop lifecycle state machine (§8).
`packages/signalflow`	SignalFlow routing (§12.4).
`packages/reputation`	Per-domain reputation, decay, anti-Sybil scoring (§10, §11).
`packages/dag-substrate`	Substrate (§13). Append-only event log.
`packages/identity`	Identity layer (§11.1–11.3). OAuth, on-device KYC, DID, ZKP.
`packages/psll-sync`	PSLL anchoring service (§11.4). Merkle commitments to DAG.
`packages/quest-market`	Quest marketplace (§12).
`packages/validation-neighborhoods`	Sharded micro-validation routing (§9, §12.5).
`packages/dfao-registry`	DFAO registry (§15).
`packages/governance`	Proposals, conviction voting, threshold execution (§15).
`packages/temporal`	Seasons, decay scheduling, loop timeouts.
`packages/token-economy`	XP, CT, and related access-threshold mechanics.
`packages/credentials`	Verifiable credential issuance and verification helpers.
`packages/epistemology-engine`	Emergent peer-review witness layer (§9.5). Read-mostly. Multiple instances may run independently.
`packages/github-parasite`	v0.1 scaffold. Bridges GitHub App events into the informational-domain contribution graph. Explicitly satisfies the three Non-Extraction tests.
`packages/contracts`	Shared types and schemas, single source of truth for cross-package interfaces.

Any of these packages can be replaced by an alternative implementation in another language, provided the replacement satisfies `docs/PROTOCOL.md v0.1`. The `contracts` package is the single source of truth for cross-package interfaces at the reference-implementation layer; a conformant alternative implementation must ship its own equivalent.

18.1 What the reference implementation does not commit the protocol to

Database engines, message buses, programming languages, transport, deployment topology, storage backends, and observability stack are all reference-implementation concerns, not protocol concerns. The protocol says nothing about them. See `docs/PROTOCOL.md` §1.

19. Open Engineering Gaps

There are 65 identified engineering gaps across 13 categories, enumerated in `docs/GAPS.md`. The most significant open items:

P1 (blockers for phase 2). Quorum size formula for variable-domain rings. Validator collusion detection thresholds. Cartel threshold formal analysis at >50% domain reputation. Wash-loop detection across colluding identities. Bribery resistance under IT decay. Funded-validator (corporate-capture) defenses. 4-factor SignalFlow weight tuning. Cold-start validator bootstrapping. Geographic and language balancing in SignalFlow. Sybil-resistant load distribution under burst traffic. ΔS unit harmonization across the eight domains (the F1 falsifier in operational form). Falsification-condition specifications for the cognitive, social, and governance domains. Calibration drift detection. Verdict vocabulary standardization (canonical affirmative verdict values and API field naming consistency).

P2 (robustness and security). Causal-edge gossip protocol specification. Partition tolerance and merge rules. DAG garbage collection and pruning policy. Replay attack protection. PSLL-anchor receipt cadence. Retroactive validation edge cases under validator churn. Burn-cascade limits when one loop's burn invalidates dependents. Settlement reliability under network partition. Retro-validation incentive structure. DFAO migration hand-off protocol. Quorum loss recovery for micro-tier DFAOs. Conflicting proposals across nested DFAOs. Influence-decay edge cases on dormant members. Cross-tier proposal escalation rules. ZKP scheme final selection. Selective-reveal threshold mechanics. Nullifier collision resistance proof. PSLL selective-disclosure protocol. Cross-DFAO data isolation.

P3 (ecosystem maturity). Skill DAG design. Oracle integration protocol. Performance and scalability targets. Migration and upgrade paths for future protocol versions.

Gaps are not failures. They are the engineering backlog. Acknowledging incompleteness is a prerequisite for systematic completion, and it is the only honest register for a live specification.

20. Rejected Framings (In-Line Summary)

Framings that appeared in earlier versions and are formally retired live in `docs/REJECTED_FRAMINGS.md`. The full text and reasoning is there. This section summarizes.

R1. XP = ΔS / c_L² (retired in canonical-v3.1.3). Codex v1.0 taught this as an "irreducible XP" floor, implemented in `xp-mint` as `Math.max(fullXP, irreducibleXP)`. Retired on three grounds. G1: the physics analogy is wrong; dividing an entropy delta by the square of a substrate speed borrows the form of `E = mc²` without borrowing any of the derivation. G2: the stated purpose (guaranteeing legitimate loops mint something positive) was patching around the raw-seconds bug in the settlement-time factor at the wrong layer. G3: subtractive credibility; publishing an equation with no derivation invites competent readers to search for one, find none, and conclude the rest of the spec is at the same level. Replacement: the v3.1.3 formula, which is bounded and non-negative by construction, so no "irreducible" branch is needed.

R2. "Cryptographic loop closure" as a security guarantee (reserved). Codex v1.0 §7 language implied loops close under a cryptographic invariant. In practice they close under a validator-neighborhood invariant plus an audit trail. A subtractive rewrite is pending in a follow-up docs PR.

R3. "Trilemma theorem" as proved (reserved). The v_min / V4-6 phrasing in Codex v1.0 Appendix J was presented as a theorem but is a conjecture. The retirement path is: restate it as an open conjecture with current partial results as lemmas, or produce a full proof. Not part of v3.1.3.

Codex v2.0 does not carry an in-document preservation appendix for these framings. Readers who want to know what the project used to say and no longer says should read `REJECTED_FRAMINGS.md`.

21. Roadmap and Adoption Criteria

Codex v2.0 is not a promise that the protocol is finished. It is a statement of where the specification stands at the moment of the v3.1.3 release. The near-term roadmap is defined by the P1 items in `docs/GAPS.md` and by follow-up docs work.

21.1 Near-term (within the current cycle)

- Ship the non-extraction test harness (`packages/xp-mint/tests/non-extraction.test.ts`) that exercises T1, T2, and T3 against the public surface of every mint-adjacent package.
- Land the domain-specific measurement operator M_d implementations for at least three of the eight domains, with per-domain calibration tables and validator-neighborhood recipes. Informational is the natural first domain because it maps most directly onto the reference implementation's current evidence surface.
- Reconcile the F naming collision surfaced in §10 across `SPEC_v3.1.md`, `PROTOCOL.md`, and `NORMALIZATION.md`. F remains Falsifiability; frequency-of-decay becomes F_freq.
- Close R2 and R3 in `REJECTED_FRAMINGS.md`, either by subtractive rewrite or by producing the missing derivations.
- Formalize the cartel threshold analysis from `docs/cartel-threshold-analysis.md` §9 open vulnerability, in light of the v3.1.3 T_floor.

21.2 Adoption criteria

The Codex is adopted when three conditions hold together. The protocol contract in `docs/PROTOCOL.md v0.1` is stable across two consecutive governance cycles without breaking changes. At least three domains have M_d implementations that satisfy `NORMALIZATION.md` §4 invariants under adversarial testing. The Non-Extraction test harness passes against every mint-adjacent package in the reference implementation and against at least one independent alternative implementation.

Any change to the Codex proposed after adoption is itself a governance-domain loop, and it must pass its own validator neighborhood before it takes effect.

22. Glossary and Formula Reference

Actor. An entity that can open loops, close loops, hold XP thresholds, hold CT, and be observed by validators.

b_e (bits-equivalent). The single common unit for cross-domain ΔS. All domain-native measurements are converted to b_e before entering the mint formula.

c_L (retired). Per-domain causal closure speed. Used in the retired framing XP = ΔS / c_L^2. See §20 R1.

Claim Package. The minimum interoperable payload the personal AI sends to the network. Includes a schema-conformant claim, an identity proof, a PSLL anchor reference, and optional quest-market metadata.

Contribution graph. The graph of contributions (all classes of work, including validation) that the mesh records and the `epistemology-engine` observes. Validation is a property of the graph, not a separate tier.

CT (Contribution Token). A per-actor structural coefficient that shapes access weight in ties and vote weight in validation. Non-transferable. Not spent by voting.

DAG. Directed acyclic graph. The append-only substrate that records loop state, validator verdicts, and mint events.

DFAO. Digital Fractal Autonomous Organization. The unit of governance. Nestable.

DID. Decentralized Identifier. W3C standard for actor identifiers.

E (Evidence classes, in §8). E1 independently reproducible, E2 domain-native, E3 tamper-evident.

E (Entropy vector, in §7). The eight-domain vector of measured entropy reduction per claim.

ε_d. Domain-specific tolerance on ΔS measurement across validators.

F (Falsifiability, in this Codex). The falsifiability score in the XP formula, $F \in [0, 1]$. Property of the claim.

F_freq (Frequency-of-decay). The frequency-of-decay penalty for an actor's recent same-class loops. Retained in the formula; renamed here to disambiguate from Falsifiability.

F1–F4. The four falsification conditions in `NORMALIZATION.md`. See §6.2.

KYC. Know Your Customer, in the identity sense. On-device only in the Extropy Engine.

Loop. A bounded piece of work that opens, gathers evidence, is scored, and closes. See §8.

λ_d. Per-domain settlement-decay constant. Enters $T_s = \exp(-\lambda_d \cdot \Delta t)$.

M_d. Domain-specific measurement operator that converts raw evidence to ΔS_b_e.

Non-Extraction. The architectural invariant that XP and CT never bridge to external transferable value. See §3.

PSLL. Personal Signed Local Log. Per-participant append-only provenance log, anchored to the DAG via Merkle roots. See §11.4.

Q_d. Domain-specific quorum function.

R (Rarity). Rarity coefficient in the XP formula. Domain-specific. Not reputation.

Reputation. Property of the actor over time. Lives in CT and per-domain reputation vectors. Never enters the mint formula.

ρ (rho, contribution/draw ratio). ΔS_produced / ΔS_consumed, computed on the ledger. Governs access degradation under Non-Extraction.

ρ_min_d. Domain-specific floor on ρ. Below this, access degrades.

SignalFlow. The routing function that assigns quests to participants. Four-factor weighting.

Sybil. A single actor operating multiple identities. Countered by the identity layer's uniqueness proofs and by nullifiers.

T_floor. Governance-set settlement-time floor. Default 0.01.

T_s. Normalized settlement-time factor. $T_s = \exp(-\lambda_d \cdot \Delta t)$, clamped into (T_floor, 1].

τ_action_d, τ_validator_d. Access thresholds for domain-d actions and for participating in a domain-d validator neighborhood.

Validator neighborhood. A set of actors above the τ_validator_d threshold, participating in a specific loop's closure. Per-loop, per-domain. No actor is a validator in general.

w. Domain-weight vector in the XP formula. Per-DFAO override on top of ecosystem defaults.

XP. Extropy Points. Non-transferable access threshold. Minted from verified ΔS_b_e under the canonical formula. Never a balance to be extracted.

ZKP. Zero-knowledge proof. Wraps identity credentials so the network sees proof of uniqueness without raw material.

Formula reference

```
XP = R × F × ΔS_bₑ × (w · E) × min(log(1/Tₛ), log(1/T_floor))
Tₛ = exp(-λ_d · Δt)              Δt in seconds
Tₛ ∈ (T_floor, 1]                by construction
Formula version: canonical-v3.1.3
Implementation: packages/xp-formula/src/index.ts
```

23. References and Companion Works

Repository documents referenced in this Codex (all at `github.com/00ranman/extropy-engine`):

- `docs/PROTOCOL.md` (v0.1). External protocol contract.
- `docs/NORMALIZATION.md`. b_e, M_d, and the F1 through F4 falsifier contract.
- `docs/NON_EXTRACTION.md`. The extraction invariant and three tests.
- `docs/REJECTED_FRAMINGS.md`. R1 ($XP = \Delta S / c\_L^2$) retired; R2 and R3 reserved.
- `docs/SPEC_v3.1.md`. The prior technical specification.
- `docs/GAPS.md`. 65 engineering gaps across 13 categories.
- `docs/cartel-threshold-analysis.md`. Game-theoretic analysis of validator cartels.
- `docs/GOVERNANCE_DEFAULTS.md`. Provisional defaults and vote-tier assignments.
- `docs/VALIDATION_IS_EMERGENT.md`. No validator class.
- `docs/THREE_LAYER_SEPARATION.md`. Hard rules on the three architectural layers.
- `docs/PSLL.md`. Personal Signed Local Log.
- `docs/IDENTITY.md`. Identity layer.

- `docs/QUEST_MARKET.md`. Quest marketplace.
- `docs/CONTRIBUTION_GRAPH.md`. The contribution graph as substrate for validation.
- `docs/CHANGELOG.md`. Release history including v3.1.3.
- `architecture/AUTARKY.md`. Digital Autarky as principle.

Reference implementation packages (all at `github.com/00ranman/extropy-engine/tree/main/packages`): `xp-formula`, `xp-mint`, `loop-ledger`, `signalflow`, `reputation`, `dag-substrate`, `identity`, `psll-sync`, `quest-market`, `validation-neighborhoods`, `dfao-registry`, `governance`, `temporal`, `token-economy`, `credentials`, `epistemology-engine`, `github-parasite`, `contracts`.

Companion works.

- Randall Gossett, *Unfuck the World for a Dollar* (companion book; work in progress).
- Randall Gossett, *XP Timekeeping System: Temporal DAG Infrastructure, Entropy Economics, and the Post-Calendar Coordination Problem* (2026).
- The Signal paper on representational fidelity decay (referenced in `docs/NON_EXTRACTION.md` §2.2 as the source of the Nash-flip analysis).
- Public site: `lladnaros.com`.

External references.

- Rolf Landauer, “Irreversibility and Heat Generation in the Computing Process,” IBM J. Res. Dev. 5 (1961) 183.
- Charles H. Bennett, “The Thermodynamics of Computation,” Int. J. Theor. Phys. 21 (1982) 905.
- Claude E. Shannon, “A Mathematical Theory of Communication,” Bell Sys. Tech. J. 27 (1948) 379–423, 623–656.
- W3C, Decentralized Identifiers (DIDs) v1.0.

Codex v2.0 supersedes Codex v1.0 in full. Framings v1.0 taught that v2.0 does not carry are retired in `docs/REJECTED_FRAMINGS.md`. This document is the internal specification; the external contract is `docs/PROTOCOL.md v0.1`. The system is designed to be falsifiable, not infallible. Every domain defines what would prove it wrong. That is the difference between engineering and ideology.